Exercise 3 – NLP Representation

Leonardo Antero SP. Yoro

IT Elective IV, BSIT

Jose Rizal University

Mandaluyong, Philippines

leonardoantero.yoro@my.jru.edu

***Abstract* — This exercise introduces core text representation techniques in Natural Language Processing (NLP), including dictionary lookup for word-to-ID mapping, one-hot encoding, Bag of Words (BoW), and Term Frequency-Inverse Document Frequency (TF-IDF). These methods convert raw text into numerical formats suitable for computational models. While dictionary lookup can be useful with simple-to-implement tokenization, it doesn't carry semantics, meaning, or context. One-hot encoding slightly improves this by representing words as sparse-dimension binary vectors. Both methods preserve unique word identities but may produce high-dimensional data. For example, BoW converts text to vectors based on the frequency count of words. BoW will let you know whether or not a unique word occurred in the text but ignores the order and meaning of how the words worked together. An improvement was made from BoW with TF-IDF. TF-IDF weights terms based on their importance in the assessment of relevance across a corpus. The exercise offers practical experience in implementing these techniques using Python and the scikit-learn library. The primary goal is to understand each representation method along with its inputs and outputs, laying a strong foundation for effective text preprocessing and guiding the selection of appropriate representation methods for future NLP tasks like classification, clustering, or language modeling.**

Keywords— Text Representation, NLP, One-Hot Encoding, Bag of Words, TF-IDF, Tokenization, Text Preprocessing

I. INTRODUCTION

The present exercise is devoted to examining different methods to represent text data for Natural Language Processing (NLP). The importance of representation of text is vital for the text to be converted from raw text form to numerical form that a computational model can use and interpret. The exercise will explore simple methods like a dictionary where each word is given a unique numeric ID, Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), and Word Embeddings, as there are trade-offs among the various approaches that capture alternative semantic and syntactic properties of the text.

The lookup device for dictionaries could also be viewed as a primary example of tokenization and numeric encoding, as it was also important for mapping numeric ID's to each word, which opened the door to many forms of metric-like encoding for text. One-hot encoding was exploring, as a simple example of representing words as sparse binary vectors each represented by a vector having one high (1) value for that unique ID. Completing this exercise provides practical experience with text representation techniques and fosters a clear understanding of how these functions operate, including their inputs and outputs, as well as the precise Python syntax required to implement them.

II. NATURE AND PURPOSE OF THE ACTIVITY

The exercise involved examining and implementing a range of fundamental text representation methods that are often used in NLP, specifically it looked at dictionary word-ID mapping, which provides basic tokenization and encoding, the bag of words (BoW) technique which creates frequency vectors from text, the term frequency-inverse document frequency (TF-IDF) method to modulate the frequency of words on their significance to documents, and looked at word embeddings to define semantic structures of words in a dense vector space.

The Jupyter notebook included in the task had utility for helping us to pragmatically use each of the text representation methods, which allowed us to inspect the input data, the transformations undertaken, and the output. What the documentation task asked us to do was describe the code in each code cell in detail, in terms of what each function does, syntax, inputs & outputs of functions, how the code flows through the cell, so that the users could develop a better understanding of the core NLP concepts and how they are implemented in Python.

The aim of the exercise was to provide a good overview of text representation methods, and some of the skills any practitioner would need to perform any further NLP analysis of text; such as text classification, text clustering, and creating language models.

a) Bag of Words (BoW)

The Bag of Words (BoW) model is one of the most straightforward methods in NLP for transforming text into a numerical format which can be used for machine learning algorithms. The BoW model looks at a document as a "bag" of words and does not factor in the order or arrangement of the words in the document. It instead only looks at if the word appears and/or how often it appears. The primary value of the BoW model is that is is so simple that it is relatively easy to implement and understand. It captures the minimum necessary to speak meaningfully about the document: the existence and frequency of words. It makes suitable computational assumptions about the relationship

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

of the vocabulary which are efficient and allows it to work with large size vocabularies.

The BoW model has several limitations. The biggest limitation is that the BoW model ignores the meaning and context of the words, it treats each word as an independent feature, and it ignores the relationships between words or phrases. [1]

b) Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency-Inverse Document Frequency (TF-IDF) measures the significance of a word in a document compared to a corpus. TF-IDF is made from two components: Term Frequency (TF), the number of times a given term appears in a document, and Inverse Document Frequency (IDF), how common or rare a term is across a corpus. The complete TF-IDF value is established from the computation of TF and IDF together which actually ranks documents on relevance.

TF-IDF is often employed in many settings, particularly information retrieval and document classification; yet, it has limitations (it does not take semantic meaning into account). Nevertheless, it is an important method for assessing very large text datasets. [2]

c) One-hot Encoding

One-Hot Encoding is a fairly simple text preprocessing technique in Natural Language Processing (NLP) that will create a binary vector for each word in a text corpus. Each position in the vector represents a unique word in the whole corpus. The 1 indicates that the word is associated with the document, while the 0 indicates that the word is not. This operation is simple to apply and functionally sufficient for small datasets of associated words, yet gives you a very high-dimensional vector with significant memory overhead, and lacks semantic context. One-Hot Encoding is most effective with problems that have limited vocabularies, like multi-class text classification. [3]

III. Observations

The exercise provided hands-on experience with several fundamental text representation techniques in Natural Language Processing (NLP), each with distinct characteristics and implications.

a) Dictionary Lookup

The dictionary lookup method provides a basic mapping of unique words to numeric IDs that allows straightforward tokenization and encoding of text data. This base method allows a straightforward way of indexing words with their ID numbers allowing the term to be referenced with the unique numeric ID. The example provided demonstrates how pulling unique words from sentences and dollar lookups would work by both word and ID providing a simple, two-way mapping.

Although the dictionary lookup method is relatively easy to implement, and a useful way of indexing, it does not have a meaning or retain any context of the words. Each word is treated independently of one another without regard to its relationship with other words, and it is easy to see that is a limitation for any task that relies on some structure or meaning in a language. Models, such as those for text representation, are needed to get beyond these limitations.

```python
def extract_words(sentences):
    """
    Extracts unique words from a list of sentences and assigns each a numeric ID.
    Args:
        sentences (list of str): A list of sentences.
    Returns:
        dict: A dictionary with numeric IDs as keys and words as values.
    """
    # Split sentences into words and use a set to keep unique words
    unique_words = set()
    for sentence in sentences:
        words = sentence.split()
        unique_words.update(words)

    # Create a dictionary with incrementing numeric IDs as keys
    word_dict = {i: word for i, word in enumerate(unique_words)}

    return word_dict
```

Fig. 1. Word Extraction and Numeric ID declaration

```python
def lookup_by_id(word_dict, numeric_id):
    # Iterate through the dictionary to find the word
    for id, dict_word in word_dict.items():
        if int(numeric_id) == id:
            return f"The numeric id '{numeric_id}' is used to identify the word: {dict_word}"
    return f"The ID '{numeric_id}' was not found in the dictionary."

def lookup_by_word(word_dict, word):
    # Iterate through the dictionary to find the word
    for id, dict_word in word_dict.items():
        if dict_word == word:
            return f"The word '{word}' has the numeric ID: {id}"

    return f"The word '{word}' was not found in the dictionary."
```

Fig. 2. Lookup Function

```python
while True:
  user_input = input("Enter look up key (or 'exit' to quit): ").strip()

  if user_input.lower() == 'exit':
    print("Exiting the program.")
    break

  result = lookup_by_id(word_dict, user_input)
  print(f"Result: {result}\n")
```

```
Enter look up key (or 'exit' to quit): 5
Result: The numeric id '5' is used to identify the word: not

Enter look up key (or 'exit' to quit): exit
Exiting the program.
```

Fig. 3. Interactive Lookup Loop

b) One-hot Encoding

One-hot encoding went one step beyond a dictionary by representing each word as a sparse binary vector, where each word is represented as an element. The code shows that we can think of sentences as a 3-D array where each word in a sentence will have a position in each sentence made up of a vector of 1's and 0's indicating the presence of tokens. It maintains the identity of the word but does not maintain word order (apart from index position), and

creates high-dimensional, sparse vectors that become computationally expensive for large vocabularies. Also, this representation will not preserve the semantic relationships of words.

```
print("Final Vector Representations:")
results

Final Vector Representations:
array([[[0., 1., 0., 0., 0., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0., 0., 0.],
        [0., 0., 0., 1., 0., 0., 0., 0.],
        [0., 0., 0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 1., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0.]],

       [[0., 1., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 1., 0.],
        [0., 0., 0., 1., 0., 0., 0., 0.],
        [0., 0., 0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 1.],
        [0., 0., 0., 0., 0., 0., 0., 0.]]])
```

Fig. 4. One-hot Encoding Final Vector Representations

c) Bag of Words (BoW)

The Bag of Words model converts text documents into fixed length vectors that signify the frequency of each vocabulary from the document. This model ignores grammar and word order. In the current exercise, demonstrating using scikit-learn's CountVectorizer, we followed how a corpus is tokenized and made into a sparse matrix, a matrix where almost all elements are zeros. In this sparse matrix, each row corresponds to a document from the corpus and each column is a unique word from the entire corpus' vocabulary. The Bag of Words model based on frequency is simple and works well for many text classification problems, but it can also be strongly impacted by high-frequency common words, which add little value in discrimination. Also, BoW vectors are typically in high-dimensional space which is mostly sparse. This may be detrimental for storage and computation purposes.

```
# Sample sentences
sentences = ["The cat in the hat",
"The dog in the house", "The bird in the sky"]
# Create a CountVectorizer object
vectorizer = CountVectorizer()
# Use the fit_transform method to transform the sentences into a bag of words
bow = vectorizer.fit_transform(sentences)
# Print the sentences in the corpus
print("Sentences processed: "+str(sentences)+"\n")
# Print the vocabulary (features) of the bag of words
print("Unique Words in the corpus: "+str(list(vectorizer.vocabulary_.keys())))
print("\nBag of Words Conversion:")
print(vectorizer.get_feature_names_out())
# Print the bag of words
print(bow.toarray())

Sentences processed: ['The cat in the hat', 'The dog in the house', 'The bird in the sky']

Unique Words in the corpus: ['the', 'cat', 'in', 'hat', 'dog', 'house', 'bird', 'sky']

Bag of Words Conversion:
['bird' 'cat' 'dog' 'hat' 'house' 'in' 'sky' 'the']
[[0 1 0 1 0 1 0 2]
 [0 0 1 0 1 1 0 2]
 [1 0 0 0 0 1 1 2]]
```

Fig. 5. Sentences processed to Bag of Words Conversion

d) Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF builds on the Bag of Words model by weighing term frequencies in a document with the inverse frequency of that term's occurrence across the entire corpus. This weighting scheme takes into account that common words appearing in many documents, such as stopwords, should have less weight relative to terms that are interesting and unique to one aspects of few documents. The exercise showed several examples of using scikit-learn's TfidfVectorizer to convert a batch of documents into TF-IDF vectors, while demonstrating how the method normalizes term counts and uses a logarithmic scaling of the counts (thus emphasizing word importance). We see that TF-IDF is the go-to method for information retrieval and text mining, since it considers both term frequency and document specificity in its final representation of the document, making it easier to distinguish documents. But TF-IDF, like BoW, does not account for any semantic meaning or order to words, and therefore, it is not useful for tasks requiring more complex understanding of language.

```
print("Final Text Representation:")
print("Feature Names:", tfidf_vectorizer.get_feature_names_out())
print("TF-IDF Matrix:\n", X_tfidf.toarray())

Final Text Representation:
Feature Names: ['and' 'document' 'first' 'is' 'one' 'second' 'the' 'third' 'this']
TF-IDF Matrix:
 [[0.         0.46941728 0.61722732 0.3645444  0.         0.
  0.3645444  0.         0.3645444 ]
 [0.         0.7284449  0.         0.28285122 0.         0.47890875
  0.28285122 0.         0.28285122]
 [0.49711994 0.         0.         0.29360705 0.49711994 0.
  0.29360705 0.49711994 0.29360705]]
```

Fig. 6. TF-IDF Final Text Representation and TF-IDF Matrix

IV. Discussion

This exercise reiterated the prevalence of text representation as a first step in any Natural Language Processing (NLP) pipeline. Engaging in hands-on practice and documentation, we gleaned a few specific insights:

a) Understanding Core NLP Techniques

The familiarity with the basic text representation techniques (dictionary lookup, one-hot encoding, Bag of Words (BoW), and TF-IDF) gives us an incredible amount of insight into how raw text will be computational represented.

b) Trade-offs Between Simplicity and Expressiveness

The easiest techniques like dictionary lookup and one-hot encoding are simple to apply but provide little information about word meaning or context. The BoW format and TF-IDF models improve model input quality by using frequency and weighting from importance but are still ultimately shallow representations of text.

c) Importance of Vector Dimensionality and Sparsity

Many representations of text result in high-dimensional and sparse vectors, which can be computationally and ecologically constraining. This knowing enables us to consider our preprocessing, feature selection, or out of the box more advanced embeddings.

d) Value of Python NLP-Related Libraries

Libraries like scikit-learn allow complicated transformations to take place while providing a convenient API for quickly experimenting and measuring results. Understanding the parameters and outputs of libraries is essential for using them for NLP tasks.

e) Documentation enhances Comprehension

Writing out sufficient information about the functionality of the code, as well as its inputs, outputs, and flow helped me to understand how each method functions internally, and the interface between theory and practice of how each method operates in an NLP application.

f) Foundation for Advanced NLP

Mastering these basic text representation techniques provides a solid foundation to explore advanced methods such as word embeddings, language models, and neural network-based NLP approaches, which effectively address the limitations of traditional techniques in capturing semantic and contextual relationships.

This exercise also demonstrated both the strength of traditional text representation methods, as well as their weaknesses. The results also further highlight the importance of selecting the selection of an applicable method based on both the task, properties of data, and computational limitations.

V. Conclusion

The following exercise presented a broad overview of the primitive text representation methods critical to Natural Language Processing (NLP). This overview demonstrated dictionary lookup, one-hot encoding, Bag of Words, and TF-IDF and how all types of raw text can be changed into numerical formats that can be input into a computational model. Each method had trade-off considerations between ease and computational cost, and the ability to convey semantic meaning.

Despite their limitations, these foundational methods play a crucial role in text data preprocessing for various NLP tasks, such as classification, clustering, and information retrieval. As practitioners will now know how to properly prepare text data, they can apply their judgement for identifying the right representation method, taking the consideration and preferences afforded by the context and demands of their project into consideration, while providing quality and timely solutions in the end-to-end pipeline for NLP applications.

VI. References

[1] T. MistrY, “Bag of Words: A Beginner’s Guide to Text Representation,” Medium, Mar. 31, 2024. [Online]. Available: https://medium.com/@mistrytejasm/bag-of-words-a-beginners-guide-to-text-representation-f14a1f155ea2 [Accessed: Aug. 9, 2025].

[2] S. Piduguralla, “TF-IDF: Understanding the Power of Text Representation,” Medium, Jul. 30, 2023. [Online]. Available: https://medium.com/@tejaswaroop2310/tf-idf-understanding-the-power-of-text-representation-26582883410a [Accessed: Aug. 9, 2025].

[3] G. Kovalchuk, “Decoding Text Preprocessing: A Comparison of One-Hot Encoding, Bag of Words, TF-IDF, and Word2Vec,” Medium, Oct. 30, 2024. [Online]. Available: https://medium.com/@goldengrisha/decoding-text-preprocessing-a-comparison-of-one-hot-encoding-bag-of-words-tf-idf-and-word2vec-ec10782b15c7. [Accessed: Aug. 9, 2025].